板子新增内容 · 2026-09-12

本周(9/6 - 9/12)新增与更新的 8 个算法, 供打印学习。完整板子见 banzi_ICPC 网络赛版.pdf。

区间合并 **2**
MEX **4**
SPFA **5**
Bellman-Ford **9**
DAG 最短路 **11**
无向图找环 **14**
Tarjan 强连通分量 **18**
树的直径 **20**

基础算法

区间合并

所属分类: 基础算法 › 区间合并 复杂度: 合并 $O(n\ log\ n)$(输入有序 $O(n)$) / 查询 $O(log\ n)$
适用问题

- 合并重叠(或相邻)区间
- 查询下标 x 属于哪个合并区间及区间内偏移量
- 按排名快速取第 k 个合并区间的左右边界
- 在线顺序扩展可达区间

区间合并常用于查找下标所在区间及相对位置, 在图论、DP、扫描线中都很常见。
1.基础静态合并（离线预处理）
给定 n 个区间 [l, r]，合并所有重叠（或相邻）的区间。 核心思路：按左端点排序，线性扫描维护当前合并块的 [L, R]。 若输入区间保证有序, 时间复杂度 O(n); 否则 O(n log n)。
2.静态区间的高效查询
合并后得到 m 个互不相交的区间，存放在 merged 数组（下标从 0 开始）。
查询下标 x 属于哪个合并区间，以及在该区间内的偏移量（第几位）。
给定排名 k（区间一、区间二…），快速输出该区间的 [l, r]。
因为合并后的左端点严格递增，我们可以提取出所有的 L_i，利用 upper_bound 二分定位。

```
#include <bits/stdc++.h>
#define endl '\n'
using i64 = long long;
using namespace std;

vector<pair<i64, i64>> merge_seg(vector<pair<i64, i64>> &seg) { //合并并且返回合并好的区间数组
  vector<pair<i64, i64>> res;
  if(seg.empty()) {
    return res;
  }
  sort(seg.begin(), seg.end()); // 按 l 排序, l 相同按 r

  i64 L = seg[0].first, R = seg[0].second;
  for(auto &i:seg) {
    auto[l, r] = i;
    // 如果题意是“区间覆盖连续整数”（如 [1,2] 和 [3,4] 算连通),
    // 把 if (l > R) 改成 if (l > R + 1) 即可。
    if(l > R + 1) {
      res.push_back({L, R});
      L = l, R = r;
    } else {
      R = max(r, R);
    }
  }
  res.push_back({L, R});
  return res;
}

pair<i64, i64> get_edge(i64 k, vector<pair<i64, i64>> &fin_seg) {//输入k 返回第k个合并区间的左右边界
  return {fin_seg[k - 1].first, fin_seg[k - 1].second};
}

//查询下标 x属于哪个合并区间 以及在该区间内的偏移量(第几位)
pair<i64, i64> get_pos(i64 x, vector<pair<i64, i64>> &fin_seg) {
  vector<i64> seg_L;//若要多次查询 可以把这个的初始化放在外面 优化时间
  for(auto &i:fin_seg) {
```

```
    seg_L.push_back(i.first);
  }
  // 找到第一个左端点 > x 的位置
  i64 idx = upper_bound(seg_L.begin(), seg_L.end(), x) - seg_L.begin() - 1;
  if(idx < 0 || idx >= fin_seg.size()) return {-1, -1};
  if(x > fin_seg[idx].second) return {-1, -1};
  return {idx, x - fin_seg[idx].first};//区间 相对位置
}

int main() {
  int N;
  cin >> N;
  vector<pair<i64, i64>> seg(N); //原始区间 记录l, r
  for(auto &i:seg) {
    cin >> i.first >> i.second;
  }
  vector<pair<i64, i64>> fin_seg = merge_seg(seg);
  i64 ans = 0;
  for(auto i:fin_seg) {
    ans += i.second - i.first + 1;
  }
  cout << ans;

}
```

在线版(顺序扩展可达区间)

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

// 在线维护：初始位置在 x，顺序读取 m 个区间
void solve() {
    i64 n, x, m;
    cin >> n >> x >> m;

    i64 L = x, R = x; // 初始可达范围只有一个点

    for (int i = 0; i < m; i++) {
        i64 l, r;
        cin >> l >> r;

        // 关键判断：两个闭区间 [L,R] 和 [l,r] 是否有交集
        // 有交集的条件：l <= R && L <= r
        if (l <= R && L <= r) {
            L = min(L, l); // 左边界只能向左扩
            R = max(R, r); // 右边界只能向右扩
        }
    }

    cout << R - L + 1 << '\n'; // 输出最终可达区间长度
}
```

MEX

所属分类*:* 基础算法 › *MEX* 复杂度*:* -

适用问题

- 求一个集合(或多重集合)中最小的没有出现过的非负整数

定义

MEX(Minimum EXcluded)是指一个集合(或多重集合)中最小的没有出现过的非负整数。

例如:

- MEX($\{0, 1, 3\}$) $= 2$, 因为 0、1 出现了, 2 没出现
- MEX($\{1, 2, 3\}$) $= 0$, 因为 0 没出现
- MEX($\emptyset$) $= 0$, 空集没有元素, 最小的非负整数 0 没出现

图论

SPFA

所属分类: 图论 › 最短路 › 单源最短路 › 存在负权边 复杂度: 平均 $O(kE)$, 最坏 $O(VE)$
适用问题
- 含负权边的单源最短路
- 判断负环(单源版/全源版/边数法)
- 最短路径回溯

SPFA 是 Bellman-Ford 的进阶版: 对于一条边, 如果它的前驱节点根本没更新, 那这个节点连接的所有结点根本就没必要再看, 这样可以节省时间复杂度。
具体上, 维护一个队列: 对于更新了的结点, 入队它的相邻结点即可。
但竞赛中经常会被卡, 被卡特征(满足越多越危险):
- 图是网格状
- 边权差异大且方向性强
- 存在长链 + 大量旁支
- V 很大且 E 接近 O(V)

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'
const i64 INF = (1LL << 31) - 1; // 题目要求的上限 依据题目更改

// ==================== SPFA 单源版（入队次数判负环） ====================
// 返回 1 表示存在负环（源点可达），0 表示正常
bool spfa(vector<vector<pair<i64, i64>>> &Node, vector<i64> &dis,
          vector<i64> &pre_node, vector<i64> &cnt, int n, int start) {

    vector<bool> inq(n + 1, false);
    queue<i64> q;

    dis[start] = 0;
    q.push(start);
    inq[start] = true;
    cnt[start] = 1;

    while (!q.empty()) {
        i64 u = q.front();
        q.pop();
        inq[u] = false;

        if (dis[u] == INF) continue;

        for (auto &[v, w] : Node[u]) {
            if (dis[u] + w < dis[v]) {
                dis[v] = dis[u] + w;
                pre_node[v] = u;

                if (!inq[v]) {
                    inq[v] = true;
                    q.push(v);
                    cnt[v]++;
                    if (cnt[v] >= n) {
                        return 1; // 存在负环
                    }
```

```
                }
            }
        }
    }
    return 0;
}

// ==================== SPFA 全源版（检测任意位置的负环） ====================
// 返回 1 表示存在负环，0 表示正常
bool spfa_all(vector<vector<pair<i64, i64>>> &Node, vector<i64> &dis,
              vector<i64> &pre_node, vector<i64> &cnt, int n) {

    vector<bool> inq(n + 1, true);
    queue<i64> q;

    for (int i = 1; i <= n; i++) {
        dis[i] = 0;   // 超级源点：所有点距离初始为 0
        cnt[i] = 1;
        q.push(i);
    }

    while (!q.empty()) {
        i64 u = q.front();
        q.pop();
        inq[u] = false;

        for (auto &[v, w] : Node[u]) {
            if (dis[u] + w < dis[v]) {
                dis[v] = dis[u] + w;
                pre_node[v] = u;

                if (!inq[v]) {
                    inq[v] = true;
                    q.push(v);
                    cnt[v]++;
                    if (cnt[v] > n) { // 全源版阈值是 > n
                        return 1;
                    }
                }
            }
        }
    }
    return 0;
}

// ==================== SPFA 边数法版（更稳健，推荐） ====================
// 返回 1 表示存在负环，0 表示正常
bool spfa_len(vector<vector<pair<i64, i64>>> &Node, vector<i64> &dis,
              vector<i64> &pre_node, vector<i64> &len, int n, int start) {

    vector<bool> inq(n + 1, false);
    queue<i64> q;

    dis[start] = 0;
    len[start] = 0;
```

```
    q.push(start);
    inq[start] = true;

    while (!q.empty()) {
        i64 u = q.front();
        q.pop();
        inq[u] = false;

        if (dis[u] == INF) continue;

        for (auto &[v, w] : Node[u]) {
            if (dis[u] + w < dis[v]) {
                dis[v] = dis[u] + w;
                pre_node[v] = u;
                len[v] = len[u] + 1;

                if (len[v] >= n) {
                    return 1; // 路径边数达到 n，必有负环
                }

                if (!inq[v]) {
                    inq[v] = true;
                    q.push(v);
                }
            }
        }
    }
    return 0;
}

// ==================== 路径回溯 ====================
vector<i64> get_path(vector<i64> &pre_node, i64 start, i64 end) {
    vector<i64> res;
    if (end < 1 || end >= (i64)pre_node.size()) return res;

    i64 cur = end;
    while (cur != -1) {
        res.push_back(cur);
        if (cur == start) { // 关键：遇到 start 就停
            reverse(res.begin(), res.end());
            return res;
        }
        cur = pre_node[cur];
    }
    cout << "不可达" << endl;
    return {};
}

// ==================== 主函数 ====================
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
```

```
        i64 n, m;
        cin >> n >> m;

        vector<vector<pair<i64, i64>>> Node(n + 1);
        vector<i64> pre_node(n + 1, -1);
        vector<i64> dis(n + 1, INF);
        vector<i64> cnt(n + 1, 0);   // 入队次数法用
        // vector<i64> len(n + 1, 0); // 边数法用

        for (int i = 1; i <= m; i++) {
            i64 u, v, w;
            cin >> u >> v >> w;
            Node[u].push_back({v, w});
            // 无向图额外加：Node[v].push_back({u, w});
        }

        // ===== 选择一种调用方式 =====

        // 【1】单源 SPFA（从 1 号点出发），入队次数判负环
        if (spfa(Node, dis, pre_node, cnt, n, 1)) {
            cout << "YES" << endl; // 有负环
        } else {
            cout << "NO" << endl;
            // 最短路输出示例：
            // for (int i = 1; i <= n; i++) {
            //     if (dis[i] == INF) cout << "INF ";
            //     else cout << dis[i] << " ";
            // }
            // cout << endl;
        }

        // 【2】全源 SPFA（检测任意位置负环）
        // if (spfa_all(Node, dis, pre_node, cnt, n)) {
        //     cout << "YES" << endl;
        // } else {
        //     cout << "NO" << endl;
        // }

        // 【3】边数法 SPFA（更稳健）
        // if (spfa_len(Node, dis, pre_node, len, n, 1)) {
        //     cout << "YES" << endl;
        // } else {
        //     cout << "NO" << endl;
        // }
    }
    return 0;
}
```

Bellman-Ford

所属分类: 图论 › 最短路 › 单源最短路 › 存在负权边 复杂度: $O(VE)$

适用问题

- 含负权边的单源最短路
- 对经过边数有限制的最短路
- 判断负环(末轮仍在松弛)

有向无环图不管正负权, 都可以用拓扑序 + DP 求最短路。 但对于有环图, 我们无法确认当前的拓扑序, 就只能用最笨的方法: 对每条边遍历 n - 1 轮。 原因是第 k 轮后, 可以确定所有"最多 k 条边"的最短路径。
比如图 a -> b -> c -> d, 最坏情况按 c-d、b-c、a-b 的顺序遍历, 每轮只能多确定一条边的松弛, 所以最坏需要 n - 1 轮。 Bellman-Ford 本质就是一个不确定拓扑序顺序的暴力遍历每条边, 和拓扑序 + DP 求最短路类似。
Bellman-Ford 记录两个数组: 距离数组(起点距离为 0, 其他为正无穷)和前驱节点数组(用来回溯路径)。
每一轮都要遍历全部 E 条边, 因此总操作次数为 $(V - 1) \times E$, 即 $O(V \cdot E)$。
补充说明: 稠密图 E 接近 V^2, 复杂度约为 $O(V^3)$; 稀疏图 E 接近 V, 复杂度约为 $O(V^2)$。
对于 Bellman-Ford, 存在以下几种情况:
1. 提前发现距离数组不再变化, 就直接停止。
2. 最后一轮后还在变, 就再跑一次: 不再变化即有解; 还在变小则存在负权环(在环上能一直变小), 视为无解。

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'
const i64 INF = (1LL << 31) - 1;//题目要求的上限 依据题目更改

bool bellman_ford(vector<vector<pair<i64, i64>>> &Node, vector<i64> &dis,
                  vector<i64> &pre_node, int n) {
  bool stop = 0;
  for(int i = 0; i < n - 1; i++) {
    if(stop) return 0;
    stop = 1;
    for(int j = 1; j <= n; j++) {
      if (dis[j] == INF) continue;
      for(auto &i:Node[j]) {
        auto [u, w] = i;
        if(dis[j] + w < dis[u]) {
          dis[u] = dis[j] + w;
          pre_node[u] = j;
          stop = 0;
        }
      }
    }
  }

  //再进行一次测试 看看最后还变不变 还变就代表有负权环
  stop = 1;
  for(int j = 1; j <= n; j++) {
    if (dis[j] == INF) continue;
    for(auto &i:Node[j]) {
      auto [u, w] = i;
      if(dis[j] + w < dis[u]) {
        stop = 0;
      }
    }
  }
  if(!stop) {
```

```
    return 1;
  }
  return 0;

}

vector<i64> get_path(vector<i64> &pre_node, i64 start, i64 end) {
    vector<i64> res;
    if (end < 1 || end >= (i64)pre_node.size()) return res;

    i64 cur = end;
    while (cur != -1) {
        res.push_back(cur);
        if (cur == start) {                 // 关键：遇到 start 就停
            reverse(res.begin(), res.end());
            return res;
        }
        cur = pre_node[cur];
    }
    // 走到 -1 还没遇到 start，说明不可达
    cout << "不可达" << endl;
    return {};
}

int main() {
  int T;
  cin >> T;
  while (T--) {

    i64 n, m;
    cin >> n >> m;
    vector<vector<pair<i64, i64>>> Node(n + 1);
    vector<i64> pre_node(n + 1, -1);
    vector<i64> dis(n + 1, INF);
    dis[1] = 0;
    for(int i = 1; i <= m; i++) {
      i64 u, v, w;
      cin >> u >> v >> w;
      if(w >= 0) Node[v].push_back({u, w});
      Node[u].push_back({v, w});

    }

    if(bellman_ford(Node, dis, pre_node ,n)) {
      cout << "YES" << endl;
    } else cout << "NO" << endl;


  }


}
```

DAG 最短路

所属分类: 图论 › 最短路 › 单源最短路 › 有向无环图_正负均可 复杂度: $O(V + E)$

适用问题

- 有向无环图上正负权均可的最短路 / 最长路 / 路径计数(拓扑序 + DP)

对于有向无环图, 不管正负权都可以用拓扑序 + DP 求最短路, 时间复杂度为 O(V + E)。

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'

struct Edge {
    int to, w;
};

// 返回拓扑序，如果图有环则返回空 vector
vector<int> topo_sort(int n, vector<vector<Edge>> &adj, vector<int> &in_deg) {
    vector<int> res;
    queue<int> q;
    for (int i = 1; i <= n; i++) {
        if (in_deg[i] == 0) q.push(i);
    }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        res.push_back(u);
        for (auto &e : adj[u]) {
            if (--in_deg[e.to] == 0) {
                q.push(e.to);
            }
        }
    }
    if ((int)res.size() != n) return {}; // 有环
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<Edge>> adj(n + 1);
    vector<int> in_deg(n + 1, 0);

    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;  // 如果无权，读 u,v 后 w=1
        adj[u].push_back({v, w});
        in_deg[v]++;
    }

    vector<int> topo = topo_sort(n, adj, in_deg);
    if (topo.empty()) {
        cout << "存在环，无法拓扑排序" << endl;
```

```
        return 0;
    }

    // 根据题目修改 DP 初始化
    vector<i64> dp(n + 1, 0);  // 示例：最长路，初始为 0，源点可设为 0
    // 如果需要每个点至少为 1，改成 vector<i64> dp(n + 1, 1);

    for (int u : topo) {
        for (auto &e : adj[u]) {
            int v = e.to, w = e.w;
            // 修改转移方程：
            // 最长路：dp[v] = max(dp[v], dp[u] + w);
            // 最短路：dp[v] = min(dp[v], dp[u] + w);
            // 路径计数：dp[v] += dp[u];
            dp[v] = max(dp[v], dp[u] + w);
        }
    }

    // 按题目要求输出
    for (int i = 1; i <= n; i++) {
        cout << dp[i] << endl;
    }

    return 0;
}
```

例题(DAG 最长链)
(注: 仅示意拓扑序 + DP, 未输出答案)

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'

vector<int> topo_sort(vector<vector<int>> &Node, vector<int>& in_deg) {
  vector<int> res;
  queue<int> q;
  for(int i = 1; i < Node.size(); i++) {
    if(in_deg[i] == 0) q.push(i);
  }
  while (!q.empty()) {
    int cur = q.front();
    q.pop();
    res.push_back(cur);
    for(auto &i:Node[cur]) {
      if(--in_deg[i] == 0) {
        q.push(i);
      }
    }
  }
  return res;
}

int main() {
  int n, m;
  cin >> n >> m;
```

```
  vector<vector<int>> Node(n + 1);
  vector<int> in_deg(n + 1, 0);
  for(int i = 1; i <= m; i++) {
    int u, v;
    cin >> u >> v;
    Node[u].push_back(v);
    in_deg[v]++;
  }
  vector<int> topo = topo_sort(Node, in_deg);
  vector<int> dis(n + 1, 1);
  for(auto &i:topo) {
    for(auto &j:Node[i]) {
      dis[j] = max(dis[j], dis[i] + 1);
    }
  }


}
```

无向图找环

所属分类: 图论 › 环问题 复杂度: $O(V+E)$(并查集版 $O(E\,\alpha(V))$)
适用问题
- 判断无向图是否有环
- 输出环上的节点路径(DFS 版)
- 只需判断时用并查集最快(不能输出路径)

下面三个板子都能返回是否存在结点数 ≥ 3 的环, 并尽可能给出环上的节点。

一、DFS 找环（推荐，能直接输出环路径）

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'

vector<vector<int>> Node;
vector<int> parent_;
vector<int> cycle;
vector<int> vis;
bool found = false;

void dfs(int u, int p) {
    vis[u] = 1;
    parent_[u] = p;
    for(auto &v : Node[u]) {
        if(found) return;
        if(v == p) continue;
        if(vis[v]) {
            // 找到环, 从 u 回溯到 v
            found = true;
            cycle.push_back(v);
            int cur = u;
            while(cur != v) {
                cycle.push_back(cur);
                cur = parent_[cur];
            }
            cycle.push_back(v);
            return;
        }
        if(!vis[v]) {
            dfs(v, u);
            if(found) return;
        }
    }
}

int main() {
    int n, m;
    cin >> n >> m;
    Node.assign(n + 1, {});
    vis.assign(n + 1, 0);
    parent_.assign(n + 1, -1);
    for(int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        if(u == v) continue;   // 忽略自环
```

```
        Node[u].push_back(v);
        Node[v].push_back(u);
    }
    for(int i = 1; i <= n; i++) {
        if(!vis[i]) {
            dfs(i, -1);
            if(found) break;
        }
    }
    if(!found) {
        cout << "无环" << endl;
    } else {
        cout << "有环: ";
        for(auto &x : cycle) cout << x << " ";
        cout << endl;
    }
}
```

说明:
- parent_[u] 记录 DFS 树上的父节点;
- 遇到 vis[v] && v != p 就是回边，说明有环;
- 从 u 沿着 parent_ 回溯到 v，路径就是环。

复杂度: O(V + E)

二、BFS 找环（能判断，路径略麻烦）

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'

vector<vector<int>> Node;
vector<int> parent_;
vector<int> vis;

bool bfs_check(int start) {
    queue<int> q;
    q.push(start);
    vis[start] = 1;
    parent_[start] = -1;
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        for(auto &v : Node[u]) {
            if(!vis[v]) {
                vis[v] = 1;
                parent_[v] = u;
                q.push(v);
            } else if(v != parent_[u]) {
                return true;   // 有环
            }
        }
    }
    return false;
}

int main() {
    int n, m;
```

```
    cin >> n >> m;
    Node.assign(n + 1, {});
    vis.assign(n + 1, 0);
    parent_.assign(n + 1, -1);
    for(int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        if(u == v) continue;   // 忽略自环
        Node[u].push_back(v);
        Node[v].push_back(u);
    }
    bool found = false;
    for(int i = 1; i <= n; i++) {
        if(!vis[i]) {
            if(bfs_check(i)) {
                found = true;
                break;
            }
        }
    }
    if(found) cout << "有环" << endl;
    else cout << "无环" << endl;
}
```

说明：
- BFS 在无向简单图中遇到 `vis[v] && v != parent_[u]` 就说明有环；
- 想输出环路径的话，需要额外从 `u` 和 `v` 同时向上回溯到 LCA，比 DFS 麻烦，一般检测用 BFS，输出环用 DFS。

复杂度：`O(V + E)`

三、并查集找环（最简单，但不能输出环路径）

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'

vector<int> fa;

int find(int x) {
    return fa[x] == x ? x : fa[x] = find(fa[x]);
}

bool merge(int u, int v) {
    int ru = find(u), rv = find(v);
    if(ru == rv) return false;  // 已经连通，再加边就有环
    fa[ru] = rv;
    return true;
}

int main() {
    int n, m;
    cin >> n >> m;
    fa.resize(n + 1);
    for(int i = 1; i <= n; i++) fa[i] = i;

    bool found = false;
```

```
    for(int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        if(u == v) continue;   // 忽略自环
        if(!merge(u, v)) {
            found = true;
            break;
        }
    }
    if(found) cout << "有环" << endl;
    else cout << "无环" << endl;
}
```

说明：
- 遍历每条边，如果两端已经属于同一集合，说明加入这条边会形成环；
- 简单无向图中，这样找到的环长度一定 ≥ 3（自环已忽略，重边需额外去重）；
- 不能给出环上的具体节点，只能判断有没有环。

复杂度：$O(E \cdot \alpha(V))$，近似 $O(E)$

三个板子的对比

方法	能否判断	能否输出环路径	复杂度	适用场景
DFS	能	能，回溯即可	$O(V+E)$	需要具体环，最常用
BFS	能	麻烦，需要找 LCA	$O(V+E)$	只需要判断，或者求最短环
并查集	能	不能	$O(E\ \alpha(V))$	只需要判断，代码最短

注意事项

1. **自环**：`u == v` 是长度 1 的环，题目要求 ≥ 3，所以读入时直接跳过。
2. **重边**：`u - v` 有两条边会形成长度 2 的环，也不满足 ≥ 3，需要去重。 如果题目可能有重边，可以在读入时用 `set<pair<int,int>>` 去重，或者只保留一条。
3. **非连通图**：需要遍历所有未访问的点，确保每个连通分量都检查过。
4. **多个环**：以上代码找到第一个环就返回。如果需要找所有环，要用 Tarjan 求点双 / 边双，或者 DFS 枚举。

选择建议：

“**要判断有没有环，用并查集最快最简洁；要输出环上的节点，用 DFS；BFS 适合只需要判断或求最短环的场景。**”

Tarjan 强连通分量

所属分类: 图论 › 连通性问题 复杂度: $O(V + E)$

适用问题

- 求有向图的强连通分量
- 缩点(把每个 SCC 缩成一个点)
- 缩点后在 DAG 上做 DP(如点权最长路)

```
#include <bits/stdc++.h>
using i64 = long long;
const i64 INF = 1e18;
#define endl '\n'
using namespace std;
i64 n, m;
i64 tim = 0;
i64 cnt = 0;
vector<i64> a, dfn, low, root, scc, scc_w;
vector<bool> instk;
vector<vector<i64>> Node;
stack<i64> S;
void tarjan(i64 u) {
  dfn[u] = low[u] = ++tim;//对每个刚进入的结点初始化并且入栈
  S.push(u);
  instk[u] = 1;
  for(auto &v:Node[u]) {
    if(dfn[v] == 0) {//如果dfs到的结点没有入栈 入栈
      tarjan(v);
      low[u] = min(low[u], low[v]);//自己的子结点是否有回溯路径更短
    } else if(instk[v]){
      //如果instk == 1 此时的路径是回溯结点 比对下是否更短
      //low的定义是自己的一条回溯结点 所以不要用low
      low[u] = min(low[u], dfn[v]);
    }
  }

  if(dfn[u] == low[u]) {
    cnt++;
    i64 sum = 0;
      while (true) {
          i64 v = S.top(); S.pop();
      scc[v] = cnt;
      sum += a[v];
          instk[v] = false;
          root[v] = u;
          if (v == u) break;
      }
    scc_w.push_back(sum);

  }//出栈 给每个结点缩点绑定根 并且统计总点权 标号从1开始
}

int main() {

  cin >> n >> m;
  a.assign(n + 1, 0);
  scc.assign(n + 1, 0);
```

```
  Node.resize(n + 1);
  dfn.assign(n + 1, 0);
  low.assign(n + 1, 0);
  root.assign(n + 1, 0);
  instk.assign(n + 1, 0);
  for(i64 i = 1; i <= n; i++) cin >> a[i];
  for(i64 i = 1; i <= m; i++) {
    i64 u, v;
    cin >> u >> v;
    Node[u].push_back(v);
  }
  scc_w.push_back(0); //入一个空结点 从而从1开始
  // 关键：对每个未访问节点运行 Tarjan 避免森林的存在
    for(i64 i = 1; i <= n; i++) {
        if (dfn[i] == 0) {
            tarjan(i);
        }
    }
  vector<vector<i64>> dag(cnt + 1);
  for (i64 u = 1; u <= n; u++) {
      for (auto v : Node[u]) {
          if (scc[u] != scc[v]) {
              dag[scc[u]].push_back(scc[v]);
          }
      }
  }


  vector<i64> dp(cnt + 1, 0);
    i64 ans = 0;

    for (i64 i = cnt; i >= 1; i--) {
        if (dp[i] == 0) dp[i] = scc_w[i];
        ans = max(ans, dp[i]);
        for (auto v : dag[i]) {
            dp[v] = max(dp[v], dp[i] + scc_w[v]);
        }
    }

    cout << ans << endl;
    return 0;



}
```

树的直径

所属分类: 图论 › 树上问题 复杂度: $O(n)$

适用问题

- 求非负权树的直径(最远两点的距离)
- 两次 BFS / 两次 DFS 求直径

```cpp
// 树的直径有很多种类 对于不同的种类也有不同的求法

// 1.直接的 求树的直径 并且树的直径是非负权边(>=0)
// 两次BFS/DFS 时间复杂度为O(n)

// 先对树上的任意一点u进行BFS/DFS 找到离这个点最远的v 在树的概念上 就是以一个点为根节点 找他的最深的叶节点
// 然后对找到的u/v再进行一次BFS/DFS 找到离v最远的点 这个距离就是树的直径
// 这非常的直觉 不需要怎么证明也能知道

#include <bits/stdc++.h>
using i64 = long long;
using namespace std;
#define endl '\n'

int n;
vector<vector<int>> Node;

pair<int, int> bfs(int x) { //返回最远点和距离
  vector<int> dis(n + 1, -1);
  deque<int> dq;
  int u = x;
  dq.push_back(x);
  dis[x] = 0;
  while (!dq.empty()) {
    int cur = dq.front();
    dq.pop_front();
    if (dis[cur] > dis[u]) u = cur;
    for (auto &i : Node[cur]) {
      if (dis[i] != -1) continue;
      dis[i] = dis[cur] + 1;
      dq.push_back(i);
    }
  }
  return {u, dis[u]};
}

int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);

  cin >> n;
  Node.assign(n + 1, {});
  for (int i = 1; i < n; i++) {
    int u, v;
    cin >> u >> v;
    Node[u].push_back(v);
    Node[v].push_back(u);
  }
```

```
  if (n == 1) { cout << 0 << endl; return 0; }

  auto [u, _] = bfs(1);
  auto [v, dis] = bfs(u);
  cout << dis << endl;
}
```

```
#include <bits/stdc++.h>
using i64 = long long;
using namespace std;
#define endl '\n'

int n;
vector<vector<int>> Node;
vector<int> dis;
int far;
int MAX;

void dfs(int cur) {
  if(dis[cur] > MAX) {
    MAX = dis[cur];
    far = cur;
  }
  for(auto &i : Node[cur]) {
    if(dis[i] != -1) continue;
    dis[i] = dis[cur] + 1;
    dfs(i);
  }
}

void run(int start) {
  dis.assign(n + 1, -1);
  dis[start] = 0;
  MAX = -1;
  dfs(start);
}

int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);

  cin >> n;
  Node.assign(n + 1, {});
  for (int i = 1; i < n; i++) {
    int a, b;
    cin >> a >> b;
    Node[a].push_back(b);
    Node[b].push_back(a);
  }

  run(1);
  int p = far;
  run(p);
  cout << MAX << endl;
}
```